

DFI OPCODE_ROM — Command Truth Table (JESD79-5 Table 241)

One channel, DDR5. **cmd_type** → the fixed CA H/L pattern per UI. **green** = opcode bit the ROM drives fixed; **yellow** = field position FIELD_MUX fills (row/col/BG/BA/CID/AP/BL/MRA/OP). H=1, L=0, V=valid (H or L), X=don't care. **CA1** = cycle count (L⇒2-UI, H⇒1-UI); **CA0**=L only for ACT.

1 How to read this chart

Each row = one command. A 2-cycle command uses two rows (UI 0 = first clock beat, UI 1 = second); a 1-cycle command is one row. **Columns CA0...CA13 = the 14 physical wires of the DDR5 command bus** (one shared bus; there are no separate row/col/bank pins). **Each cell = what that wire must be** for this command on this beat.

Cell values: H=drive high (1), L=drive low (0) — these are the fixed *command fingerprint* the DRAM decodes. V=valid (either level, ignored by decode). X=don't care. A **name** = that wire carries an address/attribute bit: R0..R17=row, C2..C10=column, BA0/BA1=bank, BG0..BG2=bank group, CID=chip ID, AP=auto-precharge, BL=burst length, MRA/OP=mode-reg addr/data, CW=control word, WRp=write-partial.

Colors: **green** = a fixed H/L the OPCODE_ROM hardwires (the fingerprint). **yellow** = a field the FIELD_MUX fills from the request (bank, row, column, ...).

Worked example — Read (RD), 2 cycles on the same 14 wires:

UI	CA0	CA1	CA2	CA3	CA4	CA5	CA6	CA7	CA8	CA9	CA10	CA11	CA12	CA13
0	H	L	H	H	H	BL	BA0	BA1	BG0	BG1	BG2	CID	CID	CID
1	C2	C3	C4	C5	C6	C7	C8	C9	C10	V	AP	V	V	CID

Cycle 1: green CA0..CA4 = HLHHH is the pattern that *means* “read”; at the same time bank on CA6/7, bank group on CA8–10, burst length on CA5. Cycle 2: put the **column** address on the wires, auto-precharge on CA10. Two beats, same wires reused.

How the DRAM decodes any row (the tree): (1) CA1 first — L=2-cycle, H=1-cycle. (2) CA0 — L=ACT, else keep going. (3) CA2/CA3/CA4 pick the command (e.g. within 2-cycle: CA3=L⇒mode-reg op, CA3=H⇒CAS, then CA4 = WR(L)/RD(H)). (4) latch the yellow address fields, execute. Reading a row = “green wires say *which* command; yellow wires carry the *address* for it, spread over one or two beats.”

2 2-UI commands (CA1 = L)

cmd	UI	CA0	CA1	CA2	CA3	CA4	CA5	CA6	CA7	CA8	CA9	CA10	CA11	CA12	CA13
Activate ACT	0	L	L	R0	R1	R2	R3	BA0	BA1	BG0	BG1	BG2	CID	CID	CID
	1	R4	R5	R6	R7	R8	R9	R10	R11	R12	R13	R14	R15	R16	R17
Write WR	0	H	L	H	H	L	BL	BA0	BA1	BG0	BG1	BG2	CID	CID	CID
	1	V	C3	C4	C5	C6	C7	C8	C9	C10	V	AP	WRp	V	CID
Read RD	0	H	L	H	H	H	BL	BA0	BA1	BG0	BG1	BG2	CID	CID	CID
	1	C2	C3	C4	C5	C6	C7	C8	C9	C10	V	AP	V	V	CID
Mode Reg Write MRW	0	H	L	H	L	L	MRA0	MRA1	MRA2	MRA3	MRA4	MRA5	MRA6	MRA7	V
	1	OP0	OP1	OP2	OP3	OP4	OP5	OP6	OP7	V	V	CW	V	V	V
Mode Reg Read MRR	0	H	L	H	L	H	MRA0	MRA1	MRA2	MRA3	MRA4	MRA5	MRA6	MRA7	V
	1	L	L	V	V	V	V	V	V	V	V	CW	V	V	V

WR vs RD differ *only* at CA4 (0/1) → **rw** picks it. RD/RDA and WR/WRA share UI0; **ap** is CA10 on UI1 (a FIELD_MUX bit, not the ROM). BL=CA5 UI0. MR-ops sit at CA3=L, data-ops at CA3=H.

3 1-UI commands (CA1 = H, CA0 = H)

cmd	UI	CA0	CA1	CA2	CA3	CA4	CA5	CA6	CA7	CA8	CA9	CA10	CA11	CA12	CA13
Refresh all REFab	0	H	H	L	L	H	CID3	V	V	RIR	H	L	CID	CID	CID
RFM all RFMab	0	H	H	L	L	H	CID3	V	V	V	L	L	CID	CID	CID
Refresh same-bank REFs	0	H	H	L	L	H	CID3	BA0	BA1	RIR	H	H	CID	CID	CID
RFMs same-bank RFMs	0	H	H	L	L	H	CID3	BA0	BA1	V	L	H	CID	CID	CID
Precharge all PREab	0	H	H	L	H	L	CID3	V	V	V	V	L	CID	CID	CID
Precharge same-bank PREsb	0	H	H	L	H	L	CID3	BA0	BA1	V	V	H	CID	CID	CID
Precharge per-bank PREpb	0	H	H	L	H	H	CID3	BA0	BA1	BG0	BG1	BG2	CID	CID	CID
Self-Refresh entry SRE	0	H	H	H	L	H	V	V	V	V	H	L	V	V	V
SR entry freq-change SREF	0	H	H	H	L	H	V	V	V	V	L	L	V	V	V
Power-Down entry PDE	0	H	H	H	L	H	V	V	V	V	V	H	ODT	V	V
MPC	0	H	H	H	H	L	OP0	OP1	OP2	OP3	OP4	OP5	OP6	OP7	V
NOP / PDX	0	H	H	H	H	H	V	V	V	V	V	V	V	V	V
Deselect DES	0	CS_n = H, all CA = X (deselect -- no decode)													

REF/RFM family: CA0–CA4 all 11001; variant is **CA9** (REF=H / RFM=L) and **CA10** (sb=H / ab=L), plus CA6/CA7=BA for the sb forms. PRE family: CA0–CA3 = 1101; **CA4**=pb(H)/sb·ab(L); **CA10**=sb(H)/ab(L); pb uses CA8–CA10 as BG. So the ROM drives CA9/CA10 for these, not just CA0–CA4.

4 ROM outputs (per cmd_type, drives the encoder)

output	meaning
two_ui	1 = 2-cycle (ACT/WR/RD/MRW/MRR); 0 = 1-cycle. Holds the phase 2 cycles @1:1.
cs_pattern	CS_n per UI: L on issue (DES = H). 2nd-UI CS controls non-target ODT (note 8).
ca_fix_ui0[13:0]	fixed H/L in opcode positions (green); field positions = don't-care.
ca_fix_ui1[13:0]	UI1 template for 2-UI commands.
field_sel	which fields FIELD_MUX overlays: row/col/BG/BA/CID/AP/BL/MRA/OP/WRp.

Decode summary: CA1 = 1-vs-2 cycle; CA0=L ⇒ ACT; within CA0=H,CA1=L (2-UI) CA2=H then CA3 splits MR-ops (L) from CAS (H) and CA4 splits WR (L) / RD (H); within CA0=H,CA1=H (1-UI) CA2/CA3/CA4 pick the VrefCA/REF/PRE/SRE/MPC/NOP group and CA9/CA10 pick the REF/RFM × sb/ab and PRE sb/ab variant. **ap** = UI1 CA10 for RD/WR only.